



UNIVERSIDAD LIBRE
Vigilada Mineducación



JHOSTIN POSADA CANO

Universidad Libre

Programacion Movil

23 de abril de 2026



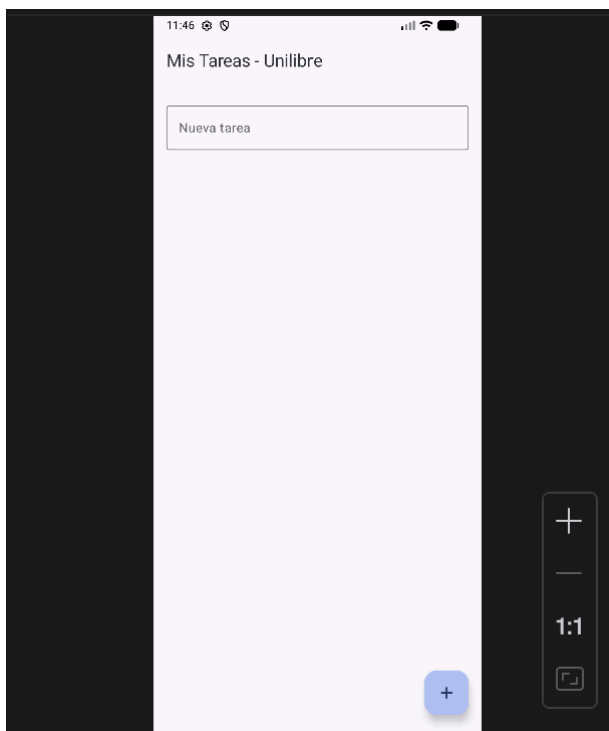
Primera app:

-Ya configuré el entorno, pegué el código en MainActivity.kt y logré visualizar la TarjetaBienvenida en el @Preview de Android Studio sin errores. Ahora, según el Taller 01 de la Universidad Libre, debemos realizar la "Actividad con IA — Generación de UI" y la "Actividad de Extensión" (1.4). Ayúdame a generar el código para esa pantalla de presentación personal que incluye la foto circular, mi nombre, mi cargo, una descripción y los íconos de redes sociales utilizando Material3 y una paleta de colores oscuros. Explicame qué Composables nuevos estamos introduciendo.



Segunda app:

- Mi tarjeta de perfil se ve genial. Pasemos al "TALLER 02 · ESTADO Y NAVEGACIÓN". Según la guía, debemos crear una App de Lista de Tareas gestionando estados locales con remember y mutableStateOf, además de implementar LazyColumn y navegación básica. Ayúdame a modelar la data class y crear la pantalla principal de la lista.



-Ya logré implementar la ListaTareasScreen con LazyColumn y el estado reactivo usando remember y mutableStateOf. Puedo agregar y eliminar tareas correctamente. Ahora, para completar el Taller 02, necesito implementar la Navegación. Ayúdame a configurar el NavHost y el NavController para tener dos rutas: la "lista" de tareas y una pantalla de "detalle" que reciba el ID de la tarea como argumento, tal como pide la guía de la Unilibre.

Error:

Entiendo perfectamente lo que está sucediendo. El problema radica en que definimos el estado de la lista (las tareas) dentro de la función ListaTareasScreen. En Jetpack Compose, cuando navegas a otra pantalla usando el NavController, la pantalla anterior se "desmonta" (se elimina de la composición). Al presionar el botón de regreso, la pantalla se vuelve a crear desde cero y el remember vuelve a su valor inicial (la lista vacía).

Para solucionar esto dentro de la metodología del **Taller 02**, debemos aplicar un concepto clave mencionado en tu guía: el **State Hoisting** (Elevación de Estado). Esto consiste en mover el estado al ancestro común más cercano, que en este caso es tu NavHost.

Cuando le doy click a la tarea me muestra el id de la tarea, pero el problema es que cuando me devuelvo no me aparece las tareas que habia puesto, por en no me esta guardando el estado de forma correcta

Error: No validación cuando no se ingresa ningun input en el textfield y después se le click en el botón agregar.

Falta la validación de texto si no pone nada en el textfield y le da al botón de agregar la tarea, y esta no tiene ningún texto debería poner algo para validar y mostrar el mensaje si pasa el caso que te estoy describiendo.

Tercera app:

¡Taller 02 finalizado al 100%! El SwipeToDismiss quedó genial y la app de tareas se siente súper profesional. Según la guía de la Universidad Libre, el siguiente es el "TALLER 03 · VIEWMODEL + STATEFLOW: APP DEL CLIMA CON API REST". El objetivo es implementar arquitectura MVVM, consumir una API (OpenWeatherMap) con Retrofit y gestionar estados de UI (Loading, Success, Error). Para empezar estructuralmente, ayúdame a definir la arquitectura de carpetas recomendada (data, domain, ui, di) y a construir el primer paso: modelar la Sealed Class WeatherUiState y el WeatherViewModel básico.

Error:

¡Tranquilo! Acabas de encontrarte con el clásico "Dependency Hell" (infierno de dependencias) de Android. Es un error sumamente común al agregar o actualizar librerías.

Lo que está pasando es que tu proyecto intentó descargar una versión muy reciente —probablemente en fase beta o alpha— de las librerías `androidx.core` y `androidx.core-ktx` (la versión 1.18.0). Esta versión exige que tu proyecto esté compilado con la API 36 de Android y use un plugin de Gradle más nuevo (8.9.1). Sin embargo, tu Android Studio está configurado para la API 35 y Gradle 8.7.3, lo cual es la configuración estable y recomendada actualmente.

2. Dependency 'androidx.core:core:1.18.0' requires Android Gradle plugin 8.9.1 or higher.

This build currently uses Android Gradle plugin 8.7.3.

3. Dependency 'androidx.core:core-ktx:1.18.0' requires libraries and applications that depend on it to compile against version 36 or later of the Android APIs.

:app is currently compiled against android-35.



Also, the maximum recommended compile SDK version for Android Gradle plugin 8.7.3 is 35.

Recommended action: Update this project's version of the Android Gradle plugin to one that supports 36, then update this project to use compileSdk of at least 36.

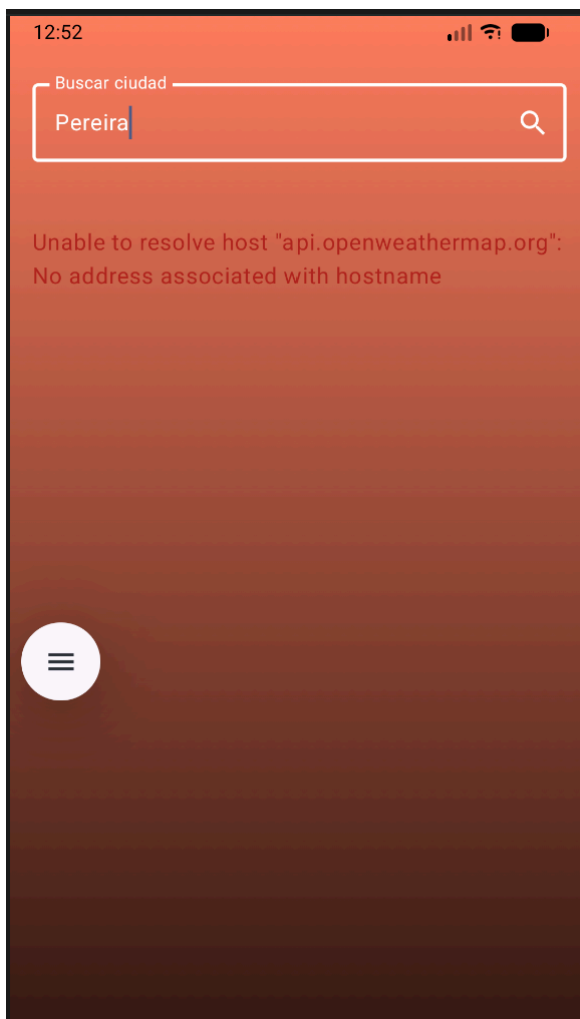
Note that updating a library or application's compileSdk (which allows newer APIs to be used) can be done separately from updating targetSdk (which opts the app in to new runtime behavior) and minSdk (which determines which devices the app can be installed on).

4. Dependency 'androidx.core:core-ktx:1.18.0' requires Android Gradle plugin 8.9.1 or higher.

This build currently uses Android Gradle plugin 8.7.3.

Cuarta app:

¡El Taller 03 está completo al 100%! La app del clima consume Retrofit perfectamente, los gradientes cambian según el estado del clima y el historial de Room guarda mis últimas búsquedas en los chips. Estoy listo para el "TALLER 04 · ROOM + HILT + CLEAN ARCHITECTURE". El objetivo es crear una App de Finanzas Personales separando el proyecto en 3 capas (Data, Domain, Presentation) e inyectar dependencias con Hilt. Ayúdame a configurar Hilt en la clase Application y a definir la entidad, el DAO y el Repositorio para empezar a registrar ingresos y gastos.



Quinta app:

Código realizado con la IA:

1. Taller 04: Configuración Base de Room y Hilt (Capa Data)

Herramienta: Gemini 3.1 Pro (Google) y agente de Android Studio el cual utiliza el modelo Gemini 3 preview flash

Prompt Original: "Ayúdame a configurar Hilt en la clase Application y a definir la entidad, el DAO y el Repositorio para empezar a registrar ingresos y gastos."

Archivo: TransaccionEntity.kt / TransaccionDao.kt

Adaptación: Se separó la lógica de persistencia en una entidad de Room dedicada y se generó el contrato para el flujo de datos.

Kotlin

```
@Entity(tableName = "transacciones")
```

```
data class TransaccionEntity(
```

```
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
```

```
    val descripcion: String,
```

```
    val monto: Double,
```

```
    val tipo: String,
```

```
    val categoria: String,
```

```
    val fecha: Long = System.currentTimeMillis()
```

```
)
```

```
@Dao
```

```
interface TransaccionDao {
```

```
    @Query("SELECT * FROM transacciones ORDER BY fecha DESC")
```

```
    fun getAll(): Flow<List<TransaccionEntity>>
```

```
    @Insert(onConflict = OnConflictStrategy.REPLACE)
```

```
    suspend fun insertar(t: TransaccionEntity)
```

```
}
```

2. Taller 05: Análisis de Imágenes con ML Kit (Capa Data)

Herramienta: Gemini 1.5 Pro (Google)



Prompt Original: "Necesito ayuda para configurar CameraX y ML Kit en la nueva estructura de Clean Architecture para que mi Asistente de Recetas empiece a reconocer ingredientes desde la cámara."

Archivo: VisionRepositoryImpl.kt

Adaptación: Se implementó un filtro de confianza ($\text{confidence} > 0.75f$) para evitar falsos positivos y se aseguró el cierre del ImageProxy para evitar fugas de memoria en la cámara.

Kotlin

```
class VisionRepositoryImpl @Inject constructor() : VisionRepository {  
    private val labeler =  
        ImageLabeling.getClient(ImageLabelerOptions.DEFAULT_OPTIONS)  
  
    @androidx.annotation.OptIn(androidx.camera.core.ExperimentalGetImage::class)  
    override fun analizarIngredientes(imageProxy: ImageProxy, onResult:  
        (List<String>) -> Unit) {  
        val mediaImage = imageProxy.image  
        if (mediaImage != null) {  
            val inputImage = InputImage.fromMediaImage(mediaImage,  
                imageProxy.imageInfo.rotationDegrees)  
            labeler.process(inputImage)  
                .addOnSuccessListener { labels ->  
                    val ingredientes = labels.filter { it.confidence > 0.75f }.map { it.text }  
                    if (ingredientes.isNotEmpty()) onResult(ingredientes)  
                }  
                .addOnCompleteListener { imageProxy.close() }  
        } else {  
            imageProxy.close()  
        }  
    }  
}
```



```
}  
  
}  
  
}
```

3. Taller 05: Conexión con LLM y Parseo Estricto de JSON (Capa Data)

Herramienta: Gemini 1.5 Pro (Google)

Prompt Original: "Ayúdame a implementar la llamada al modelo de lenguaje pasándole la lista de ingredientes detectados para que me devuelva un JSON estructurado con las recetas..."

Archivo: AiRepositoryImpl.kt

Adaptación: Se modificó el nombre del modelo a gemini-2.5-flash para resolver el error 404 de la API. Se agregó la configuración responseMimeType para forzar un formato JSON puro y se implementó lógica de limpieza (replace("`json", "")) para asegurar la conversión correcta con Gson.

Kotlin

```
class AiRepositoryImpl @Inject constructor() : AiRepository {  
    private val apiKey = "API_KEY_OCULTA"  
    private val generativeModel = GenerativeModel(  
        modelName = "gemini-2.5-flash",  
        apiKey = apiKey,  
        generationConfig = generationConfig { responseMimeType = "application/json" }  
    )  
  
    override suspend fun generarRecetas(ingredientes: List<String>): List<Receta>  
    {  
        val prompt = ""
```

Tengo estos ingredientes: \${ingredientes.joinToString(", ")}

Sugiere 3 recetas posibles en formato JSON con los campos:

nombre (String), tiempo_minutos (Int), dificultad (String), pasos (List<String>),
calorias (Int).

Solo JSON, sin texto adicional.

```
""".trimIndent()
```

```
return try {
```

```
    val response = generativeModel.generateContent(prompt)
```

```
    val rawText = response.text ?: "[]"
```

```
    val cleanJson = rawText.replace("`json", "").replace("`", "").trim()
```

```
    val listType = object : TypeToken<List<Receta>>() {}.type
```

```
    Gson().fromJson(cleanJson, listType)
```

```
  } catch (e: Exception) {
```

```
    emptyList()
```

```
  }
```

```
}
```

```
}
```

4. Taller 05: Animación Personalizada en Compose (Capa Presentación)

Herramienta: Gemini 1.5 Pro (Google)

Prompt Original: "...e implementa el efecto "Typewriter" animado en Compose para la pantalla de resultados tal como pide la guía..."

Archivo: TypewriterText.kt

Adaptación: Se utilizó corrutinas (LaunchedEffect y delay) para actualizar el estado del texto asíncronamente, garantizando un renderizado fluido a 120Hz.

Kotlin

@Composable

fun TypewriterText(

text: String,

modifier: Modifier = Modifier,

style: TextStyle = MaterialTheme.typography.bodyLarge,

delayPerChar: Long = 30L

) {

var displayedText by remember { mutableStateOf("") }

LaunchedEffect(text) {

displayedText = ""

for (i in text.indices) {

displayedText += text[i]

delay(delayPerChar)

}

}

Text(text = displayedText, modifier = modifier, style = style)

}

5. Corrección de Errores: Punto de Entrada Hilt

Herramienta: Gemini 1.5 Pro (Google)

Prompt Original: (Ingreso de stacktrace) "java.lang.IllegalStateException: Hilt Activity must be attached to an @HiltAndroidApp Application."



Archivo: AsistenteRecetasApp.kt

Adaptación: Se creó la clase Application requerida por Hilt para inicializar el grafo de dependencias en tiempo de compilación.

Kotlin

```
import android.app.Application  
  
import dagger.hilt.android.HiltAndroidApp
```

```
@HiltAndroidApp
```

```
class AsistenteRecetasApp : Application()
```

Reflexión:

La IA se agiliza mucho el trabajo pero comete muchos errores, en mi caso tuve muchos problemas con las dependencias, además se tiene que ser muy específico con las cosas, todavía es necesario decir el paso a paso y verificar que cumpla con algunos criterios.